Sri Lanka Institute of Information Technology



IT2140: Database Design and Development
Year 2, Semester 1, 2026

Assignment 01 (Part 02)

Web-Based Medicine ordering system

MediOrder: Relational Schema Design, SQL Implementation, Queries and Advanced SQL Features

Group ID: 2026-Y2-S1-MLB-B3G1-08

IT25101923	**Lakshan M.K.C.**
IT25102867	**Karunathilaka W.V.R.D.T.Y.**
IT25103946	**Vinogikan M.**
IT25101882	**Widyawansha T.D.**
IT25103725	**Hathoon M.F.S.**
IT25100979	**Kookulan E.**

TABLE OF CONTENTS

Section	Page
PART A — MAPPING EER TO RELATIONAL SCHEMA	**3**
1. Relational Schema (EER to Tables)	3
2. Primary Keys, Foreign Keys, and Constraints	5
3. ISA Mapping Justification – User Roles	6
4. Schema Refinement	6
PART B — SQL DDL IMPLEMENTATION	**7**
PART C — INSERT SAMPLE DATA	**13**
1. Sample Data INSERT Statements	13
2. Verification Queries	17
3. Created Tables (SHOW TABLES)	17
4. Sample Data Outputs	18
PART D — SQL QUERIES AND OUTPUTS	**25**
D1. Simple SELECT – Cold Chain Medicines	25
D2. JOIN – Delivery Details	25
D3. Aggregation – Prescription Verification Summary	26
D4. GROUP BY and HAVING – Medicines with Low Stock	26
D5. Subquery – Orders Above the Average Amount	27
PART E — STORED PROCEDURE	**28**
PART F — TRIGGER	**29**

Part A – Mapping EER to Relational Schema

1. Relational Schema (EER → Tables)

Table	Attributes	Primary key	Foreign keys and main constraints
users	user_id, full_name, phone_number, email, role	user_id	email UNIQUE; role limited to the seven user roles; (user_id, role) UNIQUE.
customers	user_id, role, address, date_of_birth, known_allergies	user_id	(user_id, role) → users(user_id, role); role = 'CUSTOMER'.
pharmacists	user_id, role, licence_no, specialization, shift_schedule	user_id	(user_id, role) → users(user_id, role); role = 'PHARMACIST'; licence_no UNIQUE.
pharmacy_assistants	user_id, role, packing_station, shift_schedule	user_id	(user_id, role) → users(user_id, role); role = 'ASSISTANT'.
delivery_couriers	user_id, role, vehicle_licence_no, assigned_zone, availability_status	user_id	(user_id, role) → users(user_id, role); role = 'COURIER'; vehicle_licence_no UNIQUE.
system_administrators	user_id, role, access_level	user_id	(user_id, role) → users(user_id, role); role = 'ADMIN'.
finance_managers	user_id, role, department	user_id	(user_id, role) → users(user_id, role); role = 'FINANCE'.
operations_managers	user_id, role, department	user_id	(user_id, role) → users(user_id, role); role = 'OPERATIONS'.
medicines	medicine_id, medicine_name, therapeutic_category, requires_prescription, is_cold_chain, unit_of_measure	medicine_id	Prescription and cold chain flags have default values.
inventory_batches	batch_id, medicine_id, batch_number, manufacture_date, expiry_date, quantity_available, batch_status, shelf_location,	batch_id	medicine_id → medicines; batch_number UNIQUE; expiry after manufacture; quantity ≥ 0; valid temperature range.

	min_temp_celsius, max_temp_celsius		
prescriptions	prescription_id, customer_id, prescription_type, file_url, file_hash_sha256, issue_date, upload_timestamp, verification_status	prescription_id	customer_id → customers; file_hash_sha256 UNIQUE; restricted verification statuses.
prescription_verifications	verification_id, prescription_id, pharmacist_id, action_taken, audit_reason, verification_notes, verified_at	verification_id	prescription_id → prescriptions; pharmacist_id → pharmacists; restricted actions.
chronic_refill_subscriptions	subscription_id, customer_id, medicine_id, frequency_days, next_refill_date, max_refill_count, subscription_status	subscription_id	customer_id → customers; medicine_id → medicines; frequency and maximum count > 0.
orders	order_id, customer_id, prescription_id, subscription_id, assistant_id, order_status, placed_at, total_amount, payment_method, is_chronic_refill	order_id	customer_id → customers; prescription_id → prescriptions; subscription_id → chronic_refill_subscriptions; assistant_id → pharmacy_assistants. Last three FKs may be NULL; amount ≥ 0.
order_items	order_id, item_no, medicine_id, batch_id, quantity, unit_price, scanned_serial, dispensed_at	(order_id, item_no)	order_id → orders; medicine_id → medicines; batch_id → inventory_batches; item number and quantity > 0; price ≥ 0.
delivery_routes	route_id, zone_name, gps_coordinates, geofence_polygon, estimated_distance_km, estimated_eta_minutes, is_active_zone	route_id	Estimated distance ≥ 0.
deliveries	delivery_id, order_id, courier_id, route_id, dispatched_at, delivered_at, delivery_status, otp_attempts, delivery_otp_hash, proof_photo_url, age_verified, identity_verified	delivery_id	order_id → orders and UNIQUE; courier_id → delivery_couriers; route_id → delivery_routes; OTP attempts ≥ 0.
cold_chain_tags	tag_id, medicine_id, delivery_id, min_temp_celsius, max_temp_celsius, max_transport_hours,	tag_id	medicine_id → medicines; delivery_id → deliveries; minimum temperature

	ice_pack_required, packaging_instruction, manifest_generated_at, age_verification_required, identity_verification_required		< maximum; transport hours > 0.
temperature_logs	log_id, tag_id, delivery_id, logged_at, temperature_reading, breach_detected, alert_sent, log_stage	log_id	tag_id → cold_chain_tags; delivery_id → deliveries; restricted log stages.
notifications	notification_id, user_id, delivery_id, channel_type, message_body, sent_at, delivery_status, retry_count	notification_id	user_id → users; delivery_id → deliveries. Both FKs are required; retry count ≥ 0.
audit_logs	log_id, performed_by, entity_type, action_performed, old_value, new_value, performed_at	log_id	performed_by → users.

2. PKs, FKs, and Constraints

Primary keys: Every table has a primary key to identify its records. The seven user role tables use user_id as both their primary key and a link to users. The order_items table uses the composite primary key (order_id, item_no) because one order can contain several items.

Foreign keys: Foreign keys connect related tables and prevent references to records that do not exist. For example, each prescription refers to a customer, each prescription verification refers to a prescription and a pharmacist, and each delivery refers to an order, courier and route. In order_items, medicine_id refers to medicines, while batch_id refers to inventory_batches.

Uniqueness constraints: Unique values are required for user emails, pharmacist licence numbers, courier vehicle licence numbers, prescription file hashes and inventory batch numbers. deliveries.order_id is also UNIQUE, so an order can have at most one delivery record.

Domain and validation constraints: CHECK constraints prevent negative stock, negative order amounts and invalid temperature ranges. Several status columns use defined ENUM values. The role checks and composite foreign keys prevent a user from being inserted into the wrong role table.

Optional relationships: An order may exist without a linked prescription, subscription or assigned assistant, so these foreign keys may be NULL. The current implementation requires a linked delivery for each notification.

3. ISA Mapping Justification — User Roles

We used the **shared primary key method** to map the User ISA hierarchy. The `users` table stores the common details: `user_id`, full name, phone number, email and role. The seven role tables are `customers`, `pharmacists`, `pharmacy_assistants`, `delivery_couriers`, `system_administrators`, `finance_managers` and `operations_managers`. In each role table, `user_id` is the primary key, while `(user_id, role)` is a composite foreign key referencing `users(user_id, role)`.

We keep these roles in separate tables because they have different attributes and responsibilities. For example, `customers` stores an address and date of birth, `pharmacists` stores a licence number, and `delivery_couriers` stores a vehicle licence number and availability status. This design stores common user details only once and avoids empty columns that do not apply to every role.

Disjoint roles: In this implementation, one user has one role. The `role` value in `users`, together with the role check and foreign key in each role table, ensures that a user is added to the correct subtype table. For example, a user whose role is `PHARMACIST` cannot be added to `customers`.

Matching role record: When a user is created, a row should be added to `users` and to the matching role table in the same transaction. A `NOT NULL` role value identifies the intended subtype, but **it alone does not guarantee** that the subtype row exists. Therefore, the application must complete both inserts together.

This mapping keeps the schema clear and reduces duplicate user data. It also allows each role to have its own attributes and relationships.

4. Schema Refinement

We used the composite primary key `(order_id, item_no)` in `order_items` so that each item in an order has a unique identifier. We also made `deliveries.order_id` UNIQUE to allow at most one delivery record for each order. In `orders`, `prescription_id`, `subscription_id`, and `assistant_id` may be NULL because these relationships are optional. These choices make the relational schema match the EER relationships more clearly.

Part B – SQL DDL Implementations

CREATE TABLE statement tables for all tables

The following SQL statements create the 21 tables of the MediOrder database with their primary keys, foreign keys, and constraints.

```sql
-- PART B: DDL
CREATE TABLE users (
  user_id INT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  phone_number VARCHAR(20) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  role VARCHAR(20) NOT NULL,
  CONSTRAINT chk_users_role CHECK
    (role IN ('CUSTOMER','PHARMACIST','ASSISTANT','COURIER',
              'ADMIN','FINANCE','OPERATIONS')),
  UNIQUE (user_id, role)
) ENGINE=InnoDB;

CREATE TABLE customers (
  user_id INT PRIMARY KEY,
  role VARCHAR(20) NOT NULL DEFAULT 'CUSTOMER',
  address VARCHAR(255) NOT NULL, date_of_birth DATE NOT NULL, known_allergies VARCHAR(255),
  CONSTRAINT chk_customers_role CHECK (role = 'CUSTOMER'),
  CONSTRAINT fk_customers_user FOREIGN KEY (user_id, role)
    REFERENCES users (user_id, role)
) ENGINE=InnoDB;
```

```sql
CREATE TABLE pharmacists (
  user_id INT PRIMARY KEY,
  role VARCHAR(20) NOT NULL DEFAULT 'PHARMACIST',
  licence_no VARCHAR(40) NOT NULL UNIQUE,
  specialization VARCHAR(100),
  shift_schedule VARCHAR(80),
  CONSTRAINT chk_pharmacists_role CHECK (role = 'PHARMACIST'),
  CONSTRAINT fk_pharmacists_user FOREIGN KEY (user_id, role)
    REFERENCES users (user_id, role)
) ENGINE=InnoDB;

CREATE TABLE pharmacy_assistants (
  user_id INT PRIMARY KEY,
  role VARCHAR(20) NOT NULL DEFAULT 'ASSISTANT',
  packing_station VARCHAR(40) NOT NULL, shift_schedule VARCHAR(80),
  CONSTRAINT chk_pharmacy_assistants_role CHECK (role = 'ASSISTANT'),
  CONSTRAINT fk_pharmacy_assistants_user FOREIGN KEY (user_id, role)
    REFERENCES users (user_id, role)
) ENGINE=InnoDB;
```

```sql
CREATE TABLE delivery_couriers (
  user_id INT PRIMARY KEY,
  role VARCHAR(20) NOT NULL DEFAULT 'COURIER',
  vehicle_licence_no VARCHAR(40) NOT NULL UNIQUE,
  assigned_zone VARCHAR(40),
  availability_status VARCHAR(20) NOT NULL,
  CONSTRAINT chk_delivery_couriers_role CHECK (role = 'COURIER'),
  CONSTRAINT fk_delivery_couriers_user FOREIGN KEY (user_id, role)
    REFERENCES users (user_id, role)
) ENGINE=InnoDB;

CREATE TABLE system_administrators (
  user_id INT PRIMARY KEY,
  role VARCHAR(20) NOT NULL DEFAULT 'ADMIN',
  access_level VARCHAR(30) NOT NULL,
  CONSTRAINT chk_system_administrators_role CHECK (role = 'ADMIN'),
  CONSTRAINT fk_system_administrators_user FOREIGN KEY (user_id, role)
    REFERENCES users (user_id, role)
) ENGINE=InnoDB;

CREATE TABLE finance_managers (
  user_id INT PRIMARY KEY,
  role VARCHAR(20) NOT NULL DEFAULT 'FINANCE',
  department VARCHAR(80) NOT NULL,
  CONSTRAINT chk_finance_managers_role CHECK (role = 'FINANCE'),
  CONSTRAINT fk_finance_managers_user FOREIGN KEY (user_id, role)
    REFERENCES users (user_id, role)
) ENGINE=InnoDB;
```

```sql
CREATE TABLE operations_managers (
  user_id INT PRIMARY KEY,
  role VARCHAR(20) NOT NULL DEFAULT 'OPERATIONS',
  department VARCHAR(80) NOT NULL,
  CONSTRAINT chk_operations_managers_role CHECK (role = 'OPERATIONS'),
  CONSTRAINT fk_operations_managers_user FOREIGN KEY (user_id, role)
    REFERENCES users (user_id, role)
) ENGINE=InnoDB;

CREATE TABLE medicines (
  medicine_id INT PRIMARY KEY,
  medicine_name VARCHAR(120) NOT NULL,
  therapeutic_category VARCHAR(80),
  requires_prescription BOOLEAN NOT NULL DEFAULT FALSE,
  is_cold_chain BOOLEAN NOT NULL DEFAULT FALSE,
  unit_of_measure VARCHAR(30) NOT NULL
) ENGINE=InnoDB;
```

```sql
CREATE TABLE inventory_batches (
  batch_id INT PRIMARY KEY,
  medicine_id INT NOT NULL,
  batch_number VARCHAR(50) NOT NULL UNIQUE,
  manufacture_date DATE NOT NULL,
  expiry_date DATE NOT NULL,
  quantity_available INT NOT NULL,
  batch_status ENUM('AVAILABLE','LOCKED','RECALLED') NOT NULL DEFAULT 'AVAILABLE',
  shelf_location VARCHAR(50),
  min_temp_celsius DECIMAL(5,2),
  max_temp_celsius DECIMAL(5,2),
  CONSTRAINT fk_batch_medicine FOREIGN KEY (medicine_id) REFERENCES medicines(medicine_id),
  CONSTRAINT chk_batch_dates CHECK (expiry_date > manufacture_date),
  CONSTRAINT chk_batch_quantity CHECK (quantity_available >= 0),
  CONSTRAINT chk_batch_range CHECK (min_temp_celsius IS NULL OR
    max_temp_celsius IS NULL OR min_temp_celsius < max_temp_celsius)
) ENGINE=InnoDB;

CREATE TABLE prescriptions (
  prescription_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  prescription_type VARCHAR(30) NOT NULL,
  file_url VARCHAR(255) NOT NULL,
  file_hash_sha256 CHAR(64) NOT NULL UNIQUE,
  issue_date DATE NOT NULL,
  upload_timestamp DATETIME NOT NULL,
  verification_status ENUM('PENDING','APPROVED','REJECTED') NOT NULL,
  CONSTRAINT fk_prescription_customer FOREIGN KEY (customer_id) REFERENCES customers(user_id)
) ENGINE=InnoDB;
```

```sql
CREATE TABLE prescription_verifications (
  verification_id INT PRIMARY KEY,
  prescription_id INT NOT NULL,
  pharmacist_id INT NOT NULL,
  action_taken ENUM('APPROVED','REJECTED','CLARIFICATION') NOT NULL,
  audit_reason VARCHAR(255),
  verification_notes VARCHAR(255),
  verified_at DATETIME NOT NULL,
  CONSTRAINT fk_verification_prescription FOREIGN KEY (prescription_id)
    REFERENCES prescriptions(prescription_id),
  CONSTRAINT fk_verification_pharmacist FOREIGN KEY (pharmacist_id)
    REFERENCES pharmacists(user_id)
) ENGINE=InnoDB;

CREATE TABLE chronic_refill_subscriptions (
  subscription_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  medicine_id INT NOT NULL,
  frequency_days INT NOT NULL,
  next_refill_date DATE NOT NULL,
  max_refill_count INT NOT NULL,
  subscription_status ENUM('ACTIVE','PAUSED','CANCELLED') NOT NULL,
  CONSTRAINT fk_subscription_customer FOREIGN KEY (customer_id) REFERENCES customers(user_id),
  CONSTRAINT fk_subscription_medicine FOREIGN KEY (medicine_id) REFERENCES medicines(medicine_id),
  CONSTRAINT chk_frequency CHECK (frequency_days > 0),
  CONSTRAINT chk_refill_count CHECK (max_refill_count > 0)
) ENGINE=InnoDB;
```

```sql
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  prescription_id INT NULL,
  subscription_id INT NULL,
  assistant_id INT NULL,
  order_status ENUM('PENDING_VERIFICATION','APPROVED','PACKING',
                    'DISPATCHED','DELIVERED','CANCELLED') NOT NULL,
  placed_at DATETIME NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  payment_method VARCHAR(30),
  is_chronic_refill BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT fk_order_customer FOREIGN KEY (customer_id) REFERENCES customers(user_id),
  CONSTRAINT fk_order_prescription FOREIGN KEY (prescription_id)
    REFERENCES prescriptions(prescription_id),
  CONSTRAINT fk_order_subscription FOREIGN KEY (subscription_id)
    REFERENCES chronic_refill_subscriptions(subscription_id),
  CONSTRAINT fk_order_assistant FOREIGN KEY (assistant_id)
    REFERENCES pharmacy_assistants(user_id),
  CONSTRAINT chk_order_amount CHECK (total_amount >= 0)
) ENGINE=InnoDB;

CREATE TABLE order_items (
  order_id INT NOT NULL,
  item_no INT NOT NULL,
  medicine_id INT NOT NULL,
  batch_id INT NOT NULL,
  quantity INT NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  scanned_serial VARCHAR(50),
  dispensed_at DATETIME NULL,
  PRIMARY KEY (order_id, item_no),
  CONSTRAINT fk_item_order FOREIGN KEY (order_id) REFERENCES orders(order_id),
  CONSTRAINT fk_item_medicine FOREIGN KEY (medicine_id) REFERENCES medicines(medicine_id),
  CONSTRAINT fk_item_batch FOREIGN KEY (batch_id) REFERENCES inventory_batches(batch_id),
  CONSTRAINT chk_item_number CHECK (item_no > 0),
  CONSTRAINT chk_item_quantity CHECK (quantity > 0),
  CONSTRAINT chk_item_price CHECK (unit_price >= 0)
) ENGINE=InnoDB;

CREATE TABLE delivery_routes (
  route_id INT PRIMARY KEY,
  zone_name VARCHAR(80) NOT NULL,
  gps_coordinates VARCHAR(100),
  geofence_polygon VARCHAR(255),
  estimated_distance_km DECIMAL(7,2) NOT NULL,
  estimated_eta_minutes INT,
  is_active_zone BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT chk_route_distance CHECK (estimated_distance_km >= 0)
) ENGINE=InnoDB;
```

```sql
CREATE TABLE deliveries (
  delivery_id INT PRIMARY KEY,
  order_id INT NOT NULL UNIQUE,
  courier_id INT NOT NULL,
  route_id INT NOT NULL,
  dispatched_at DATETIME,
  delivered_at DATETIME,
  delivery_status ENUM('READY','DISPATCHED','DELIVERED','FAILED') NOT NULL,
  otp_attempts INT NOT NULL DEFAULT 0,
  delivery_otp_hash CHAR(64),
  proof_photo_url VARCHAR(255),
  age_verified BOOLEAN NOT NULL DEFAULT FALSE,
  identity_verified BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT fk_delivery_order FOREIGN KEY (order_id) REFERENCES orders(order_id),
  CONSTRAINT fk_delivery_courier FOREIGN KEY (courier_id) REFERENCES delivery_couriers(user_id),
  CONSTRAINT fk_delivery_route FOREIGN KEY (route_id) REFERENCES delivery_routes(route_id),
  CONSTRAINT chk_otp_attempts CHECK (otp_attempts >= 0)
) ENGINE=InnoDB;

CREATE TABLE cold_chain_tags (
  tag_id INT PRIMARY KEY,
  medicine_id INT NOT NULL,
  delivery_id INT NOT NULL,
  min_temp_celsius DECIMAL(5,2) NOT NULL,
  max_temp_celsius DECIMAL(5,2) NOT NULL,
  max_transport_hours INT NOT NULL,
  ice_pack_required BOOLEAN NOT NULL,
  packaging_instruction VARCHAR(255),
  manifest_generated_at DATETIME,
  age_verification_required BOOLEAN NOT NULL DEFAULT FALSE,
  identity_verification_required BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT fk_tag_medicine FOREIGN KEY (medicine_id) REFERENCES medicines(medicine_id),
  CONSTRAINT fk_tag_delivery FOREIGN KEY (delivery_id) REFERENCES deliveries(delivery_id),
  CONSTRAINT chk_tag_range CHECK (min_temp_celsius < max_temp_celsius),
  CONSTRAINT chk_transport_hours CHECK (max_transport_hours > 0)
) ENGINE=InnoDB;

CREATE TABLE temperature_logs (
  log_id INT PRIMARY KEY,
  tag_id INT NOT NULL,
  delivery_id INT NOT NULL,
  logged_at DATETIME NOT NULL,
  temperature_reading DECIMAL(5,2) NOT NULL,
  breach_detected BOOLEAN NOT NULL DEFAULT FALSE,
  alert_sent BOOLEAN NOT NULL DEFAULT FALSE,
  log_stage ENUM('PACKING','IN_TRANSIT','HANDOVER') NOT NULL,
  CONSTRAINT fk_log_tag FOREIGN KEY (tag_id) REFERENCES cold_chain_tags(tag_id),
  CONSTRAINT fk_log_delivery FOREIGN KEY (delivery_id) REFERENCES deliveries(delivery_id)
) ENGINE=InnoDB;

CREATE TABLE notifications (
  notification_id INT PRIMARY KEY,
  user_id INT NOT NULL,
  delivery_id INT NULL,
  channel_type ENUM('EMAIL','SMS','WHATSAPP','IN_APP') NOT NULL,
  message_body VARCHAR(255) NOT NULL,
  sent_at DATETIME NOT NULL,
  delivery_status ENUM('QUEUED','SENT','FAILED') NOT NULL,
  retry_count INT NOT NULL DEFAULT 0,
  CONSTRAINT fk_notification_user FOREIGN KEY (user_id) REFERENCES users(user_id),
  CONSTRAINT fk_notification_delivery FOREIGN KEY (delivery_id) REFERENCES deliveries(delivery_id),
  CONSTRAINT chk_retry_count CHECK (retry_count >= 0)
) ENGINE=InnoDB;
```

```sql
CREATE TABLE audit_logs (
  log_id INT PRIMARY KEY,
  performed_by INT NOT NULL,
  entity_type VARCHAR(40) NOT NULL,
  action_performed VARCHAR(50) NOT NULL,
  old_value VARCHAR(255),
  new_value VARCHAR(255),
  performed_at DATETIME NOT NULL,
  CONSTRAINT fk_audit_actor FOREIGN KEY (performed_by) REFERENCES users(user_id)
) ENGINE=InnoDB;
```

Part C – Insert Sample Data

1. Sample Data INSERT Statements

```sql
-- PART C: SAMPLE DATA
INSERT INTO users (user_id, full_name, phone_number, email, role) VALUES
  (1, 'Sample Customer 1', '0770000001', 'user1@example.test', 'CUSTOMER'),
  (2, 'Sample Customer 2', '0770000002', 'user2@example.test', 'CUSTOMER'),
  (3, 'Sample Customer 3', '0770000003', 'user3@example.test', 'CUSTOMER'),
  (4, 'Sample Customer 4', '0770000004', 'user4@example.test', 'CUSTOMER'),
  (5, 'Sample Customer 5', '0770000005', 'user5@example.test', 'CUSTOMER'),
  (6, 'Sample Pharmacist 1', '0770000006', 'user6@example.test', 'PHARMACIST'),
  (7, 'Sample Pharmacist 2', '0770000007', 'user7@example.test', 'PHARMACIST'),
  (8, 'Sample Pharmacist 3', '0770000008', 'user8@example.test', 'PHARMACIST'),
  (9, 'Sample Pharmacist 4', '0770000009', 'user9@example.test', 'PHARMACIST'),
  (10, 'Sample Pharmacist 5', '0770000010', 'user10@example.test', 'PHARMACIST'),
  (11, 'Sample Assistant 1', '0770000011', 'user11@example.test', 'ASSISTANT'),
  (12, 'Sample Assistant 2', '0770000012', 'user12@example.test', 'ASSISTANT'),
  (13, 'Sample Assistant 3', '0770000013', 'user13@example.test', 'ASSISTANT'),
  (14, 'Sample Assistant 4', '0770000014', 'user14@example.test', 'ASSISTANT'),
  (15, 'Sample Assistant 5', '0770000015', 'user15@example.test', 'ASSISTANT'),
  (16, 'Sample Courier 1', '0770000016', 'user16@example.test', 'COURIER'),
  (17, 'Sample Courier 2', '0770000017', 'user17@example.test', 'COURIER'),
  (18, 'Sample Courier 3', '0770000018', 'user18@example.test', 'COURIER'),
  (19, 'Sample Courier 4', '0770000019', 'user19@example.test', 'COURIER'),
  (20, 'Sample Courier 5', '0770000020', 'user20@example.test', 'COURIER'),
  (21, 'Sample Admin 1', '0770000021', 'user21@example.test', 'ADMIN'),
  (22, 'Sample Admin 2', '0770000022', 'user22@example.test', 'ADMIN'),
  (23, 'Sample Admin 3', '0770000023', 'user23@example.test', 'ADMIN'),
  (24, 'Sample Admin 4', '0770000024', 'user24@example.test', 'ADMIN'),
  (25, 'Sample Admin 5', '0770000025', 'user25@example.test', 'ADMIN'),
```

Output

```
OK, 35 rows affected in 8.148ms
```

```sql
  (26, 'Sample Finance 1', '0770000026', 'user26@example.test', 'FINANCE'),
  (27, 'Sample Finance 2', '0770000027', 'user27@example.test', 'FINANCE'),
  (28, 'Sample Finance 3', '0770000028', 'user28@example.test', 'FINANCE'),
  (29, 'Sample Finance 4', '0770000029', 'user29@example.test', 'FINANCE'),
  (30, 'Sample Finance 5', '0770000030', 'user30@example.test', 'FINANCE'),
  (31, 'Sample Operations 1', '0770000031', 'user31@example.test', 'OPERATIONS'),
  (32, 'Sample Operations 2', '0770000032', 'user32@example.test', 'OPERATIONS'),
  (33, 'Sample Operations 3', '0770000033', 'user33@example.test', 'OPERATIONS'),
  (34, 'Sample Operations 4', '0770000034', 'user34@example.test', 'OPERATIONS'),
  (35, 'Sample Operations 5', '0770000035', 'user35@example.test', 'OPERATIONS');
```

```sql
INSERT INTO customers (user_id, role, address, date_of_birth, known_allergies) VALUES
  (1, 'CUSTOMER', '1 Main Street, Colombo', '1995-01-01', NULL),
  (2, 'CUSTOMER', '2 Main Street, Colombo', '1995-01-01', NULL),
  (3, 'CUSTOMER', '3 Main Street, Colombo', '1995-01-01', NULL),
  (4, 'CUSTOMER', '4 Main Street, Colombo', '1995-01-01', NULL),
  (5, 'CUSTOMER', '5 Main Street, Colombo', '1995-01-01', NULL);

INSERT INTO pharmacists (user_id, role, licence_no, specialization, shift_schedule) VALUES
  (6, 'PHARMACIST', 'PH-0006', 'Dispensing', 'Morning'),
  (7, 'PHARMACIST', 'PH-0007', 'Dispensing', 'Morning'),
  (8, 'PHARMACIST', 'PH-0008', 'Dispensing', 'Morning'),
  (9, 'PHARMACIST', 'PH-0009', 'Dispensing', 'Morning'),
  (10, 'PHARMACIST', 'PH-0010', 'Dispensing', 'Morning');

INSERT INTO pharmacy_assistants (user_id, role, packing_station, shift_schedule) VALUES
  (11, 'ASSISTANT', 'P-11', 'Morning'),
  (12, 'ASSISTANT', 'P-12', 'Morning'),
  (13, 'ASSISTANT', 'P-13', 'Morning'),
  (14, 'ASSISTANT', 'P-14', 'Morning'),
  (15, 'ASSISTANT', 'P-15', 'Morning');

INSERT INTO delivery_couriers (user_id, role, vehicle_licence_no, assigned_zone, availability_status) VALUES
  (16, 'COURIER', 'DL-0016', 'Zone 1', 'AVAILABLE'),
  (17, 'COURIER', 'DL-0017', 'Zone 2', 'AVAILABLE'),
  (18, 'COURIER', 'DL-0018', 'Zone 3', 'AVAILABLE'),
  (19, 'COURIER', 'DL-0019', 'Zone 4', 'AVAILABLE'),
  (20, 'COURIER', 'DL-0020', 'Zone 5', 'AVAILABLE');
```

Output

```
#25: OK, 5 rows affected in 6.893ms
#26: OK, 5 rows affected in 3.865ms
#27: OK, 5 rows affected in 4.196ms
#28: OK, 5 rows affected in 3.911ms
✓ SQL Script execution completed in 329ms. 4 statements executed successfully.
```

```sql
INSERT INTO system_administrators (user_id, role, access_level) VALUES
  (21, 'ADMIN', 'STANDARD'),
  (22, 'ADMIN', 'STANDARD'),
  (23, 'ADMIN', 'STANDARD'),
  (24, 'ADMIN', 'STANDARD'),
  (25, 'ADMIN', 'STANDARD');

INSERT INTO finance_managers (user_id, role, department) VALUES
  (26, 'FINANCE', 'Finance'),
  (27, 'FINANCE', 'Finance'),
  (28, 'FINANCE', 'Finance'),
  (29, 'FINANCE', 'Finance'),
  (30, 'FINANCE', 'Finance');

INSERT INTO operations_managers (user_id, role, department) VALUES
  (31, 'OPERATIONS', 'Operations'),
  (32, 'OPERATIONS', 'Operations'),
  (33, 'OPERATIONS', 'Operations'),
  (34, 'OPERATIONS', 'Operations'),
  (35, 'OPERATIONS', 'Operations');

INSERT INTO medicines (medicine_id, medicine_name, therapeutic_category, requires_prescription, is_cold_chain, unit_of_measure) VALUES
  (1, 'Sample Temperature Sensitive A', 'Therapy A', 1, 1, 'vial'),
  (2, 'Sample Temperature Sensitive B', 'Therapy B', 1, 1, 'vial'),
  (3, 'Sample Temperature Sensitive C', 'Therapy C', 1, 1, 'pack'),
  (4, 'Sample Temperature Sensitive D', 'Therapy D', 1, 1, 'box'),
  (5, 'Sample Temperature Sensitive E', 'Therapy E', 1, 1, 'vial');
```

Output

```
#29: OK, 5 rows affected in 5.213ms
#30: OK, 5 rows affected in 4.356ms
#31: OK, 5 rows affected in 3.463ms
#32: OK, 5 rows affected in 4.595ms
✓ SQL Script execution completed in 326ms. 4 statements executed successfully.
```

```
INSERT INTO inventory_batches (batch_id, medicine_id, batch_number, manufacture_date, expiry_date, quantity_available, batch_status, shelf_location, min_temp_celsius,
  (101, 1, 'B-101', '2026-01-01', '2027-01-01', 100, 'AVAILABLE', 'COLD-1', 2, 8),
  (102, 2, 'B-102', '2026-02-01', '2027-02-01', 80, 'AVAILABLE', 'COLD-2', 2, 8),
  (103, 3, 'B-103', '2026-03-01', '2027-03-01', 120, 'AVAILABLE', 'DRY-1', NULL, NULL),
  (104, 4, 'B-104', '2026-04-01', '2027-04-01', 65, 'AVAILABLE', 'DRY-2', NULL, NULL),
  (105, 5, 'B-105', '2026-05-01', '2027-05-01', 95, 'AVAILABLE', 'COLD-3', 2, 8);

INSERT INTO prescriptions (prescription_id, customer_id, prescription_type, file_url, file_hash_sha256, issue_date, upload_timestamp, verification_status) VALUES
  (1001, 1, 'DIGITAL', '/demo/prescription_1.pdf', '0000000000000000000000000000000000000000000000000000000000000001', '2026-09-10', '2026-09-11 09:00:00', 'APPROVED'
  (1002, 2, 'DIGITAL', '/demo/prescription_2.pdf', '0000000000000000000000000000000000000000000000000000000000000002', '2026-09-10', '2026-09-12 09:00:00', 'APPROVED'
  (1003, 3, 'DIGITAL', '/demo/prescription_3.pdf', '0000000000000000000000000000000000000000000000000000000000000003', '2026-09-10', '2026-09-13 09:00:00', 'APPROVED'
  (1004, 4, 'DIGITAL', '/demo/prescription_4.pdf', '0000000000000000000000000000000000000000000000000000000000000004', '2026-09-10', '2026-09-14 09:00:00', 'APPROVED'
  (1005, 5, 'DIGITAL', '/demo/prescription_5.pdf', '0000000000000000000000000000000000000000000000000000000000000005', '2026-09-10', '2026-09-15 09:00:00', 'APPROVED'

INSERT INTO prescription_verifications (verification_id, prescription_id, pharmacist_id, action_taken, audit_reason, verification_notes, verified_at) VALUES
  (1101, 1001, 6, 'APPROVED', 'Valid sample record', 'Reviewed in demo', '2026-09-11 10:00:00'),
  (1102, 1002, 7, 'APPROVED', 'Valid sample record', 'Reviewed in demo', '2026-09-12 10:00:00'),
  (1103, 1003, 8, 'APPROVED', 'Valid sample record', 'Reviewed in demo', '2026-09-13 10:00:00'),
  (1104, 1004, 9, 'APPROVED', 'Valid sample record', 'Reviewed in demo', '2026-09-14 10:00:00'),
  (1105, 1005, 10, 'APPROVED', 'Valid sample record', 'Reviewed in demo', '2026-09-15 10:00:00');

INSERT INTO chronic_refill_subscriptions (subscription_id, customer_id, medicine_id, frequency_days, next_refill_date, max_refill_count, subscription_status) VALUES
  (2001, 1, 1, 30, '2026-10-15', 12, 'ACTIVE'),
  (2002, 2, 2, 30, '2026-10-15', 12, 'ACTIVE'),
  (2003, 3, 3, 30, '2026-10-15', 12, 'ACTIVE'),
  (2004, 4, 4, 30, '2026-10-15', 12, 'ACTIVE'),
  (2005, 5, 5, 30, '2026-10-15', 12, 'ACTIVE');
```

Ln 400, Col 151

Output

```
#33: OK, 5 rows affected in 6.081ms
#34: OK, 5 rows affected in 5.168ms
#35: OK, 5 rows affected in 4.534ms
#36: OK, 5 rows affected in 3.606ms
✓ SQL Script execution completed in 378ms. 4 statements executed successfully.
```

```
INSERT INTO orders (order_id, customer_id, prescription_id, subscription_id, assistant_id, order_status, placed_at, total_amount, payment_method, is_chronic_refill) V
  (3001, 1, 1001, 2001, 11, 'DISPATCHED', '2026-09-16 11:00:00', 100.0, 'CARD', 1),
  (3002, 2, 1002, 2002, 12, 'DISPATCHED', '2026-09-17 11:00:00', 200.0, 'CARD', 1),
  (3003, 3, 1003, 2003, 13, 'DISPATCHED', '2026-09-18 11:00:00', 300.0, 'CARD', 1),
  (3004, 4, 1004, 2004, 14, 'DISPATCHED', '2026-09-19 11:00:00', 400.0, 'CARD', 1),
  (3005, 5, 1005, 2005, 15, 'DISPATCHED', '2026-09-20 11:00:00', 500.0, 'CARD', 1);

INSERT INTO order_items (order_id, item_no, medicine_id, batch_id, quantity, unit_price, scanned_serial, dispensed_at) VALUES
  (3001, 1, 1, 101, 1, 100.0, 'SERIAL-0001', '2026-09-16 13:00:00'),
  (3002, 1, 2, 102, 1, 200.0, 'SERIAL-0002', '2026-09-17 13:00:00'),
  (3003, 1, 3, 103, 1, 300.0, 'SERIAL-0003', '2026-09-18 13:00:00'),
  (3004, 1, 4, 104, 1, 400.0, 'SERIAL-0004', '2026-09-19 13:00:00'),
  (3005, 1, 5, 105, 1, 500.0, 'SERIAL-0005', '2026-09-20 13:00:00');

INSERT INTO delivery_routes (route_id, zone_name, gps_coordinates, geofence_polygon, estimated_distance_km, estimated_eta_minutes, is_active_zone) VALUES
  (4001, 'Zone 1', '6.9,79.8', 'DEMO_POLYGON', 5.0, 15, 1),
  (4002, 'Zone 2', '6.9,79.8', 'DEMO_POLYGON', 10.0, 30, 1),
  (4003, 'Zone 3', '6.9,79.8', 'DEMO_POLYGON', 15.0, 45, 1),
  (4004, 'Zone 4', '6.9,79.8', 'DEMO_POLYGON', 20.0, 60, 1),
  (4005, 'Zone 5', '6.9,79.8', 'DEMO_POLYGON', 25.0, 75, 1);

INSERT INTO deliveries (delivery_id, order_id, courier_id, route_id, dispatched_at, delivered_at, delivery_status, otp_attempts, delivery_otp_hash, proof_photo_url, a
  (5001, 3001, 16, 4001, '2026-09-16 14:00:00', NULL, 'DISPATCHED', 0, NULL, NULL, 0, 0),
  (5002, 3002, 17, 4002, '2026-09-17 14:00:00', NULL, 'DISPATCHED', 0, NULL, NULL, 0, 0),
  (5003, 3003, 18, 4003, '2026-09-18 14:00:00', NULL, 'DISPATCHED', 0, NULL, NULL, 0, 0),
  (5004, 3004, 19, 4004, '2026-09-19 14:00:00', NULL, 'DISPATCHED', 0, NULL, NULL, 0, 0),
  (5005, 3005, 20, 4005, '2026-09-20 14:00:00', NULL, 'DISPATCHED', 0, NULL, NULL, 0, 0);
```

Ln 428, Col 154

Output

```
#37: OK, 5 rows affected in 5.471ms
#38: OK, 5 rows affected in 4.465ms
#39: OK, 5 rows affected in 2.943ms
#40: OK, 5 rows affected in 6.525ms
✓ SQL Script execution completed in 345ms. 4 statements executed successfully.
```

```sql
INSERT INTO cold_chain_tags (tag_id, medicine_id, delivery_id, min_temp_celsius, max_temp_celsius, max_transport_hours, ice_pack_required, packaging_instruction, mani
  (6001, 1, 5001, 2, 8, 12, 1, 'Use tagged packaging', '2026-09-16 13:30:00', 0, 0),
  (6002, 2, 5002, 2, 8, 12, 1, 'Use tagged packaging', '2026-09-17 13:30:00', 0, 0),
  (6003, 3, 5003, 2, 8, 12, 0, 'Use tagged packaging', '2026-09-18 13:30:00', 0, 0),
  (6004, 4, 5004, 2, 8, 12, 0, 'Use tagged packaging', '2026-09-19 13:30:00', 1, 1),
  (6005, 5, 5005, 2, 8, 12, 1, 'Use tagged packaging', '2026-09-20 13:30:00', 0, 0);

INSERT INTO temperature_logs (log_id, tag_id, delivery_id, logged_at, temperature_reading, breach_detected, alert_sent, log_stage) VALUES
  (7001, 6001, 5001, '2026-09-16 15:00:00', 4.1, 0, 0, 'IN_TRANSIT'),
  (7002, 6002, 5002, '2026-09-17 15:00:00', 4.2, 0, 0, 'IN_TRANSIT'),
  (7003, 6003, 5003, '2026-09-18 15:00:00', 4.3, 0, 0, 'IN_TRANSIT'),
  (7004, 6004, 5004, '2026-09-19 15:00:00', 4.4, 0, 0, 'IN_TRANSIT'),
  (7005, 6005, 5005, '2026-09-20 15:00:00', 4.5, 0, 0, 'IN_TRANSIT');

INSERT INTO notifications (notification_id, user_id, delivery_id, channel_type, message_body, sent_at, delivery_status, retry_count) VALUES
  (8001, 1, 5001, 'SMS', 'Order 3001 dispatched', '2026-09-16 14:05:00', 'SENT', 0),
  (8002, 2, 5002, 'SMS', 'Order 3002 dispatched', '2026-09-17 14:05:00', 'SENT', 0),
  (8003, 3, 5003, 'SMS', 'Order 3003 dispatched', '2026-09-18 14:05:00', 'SENT', 0),
  (8004, 4, 5004, 'SMS', 'Order 3004 dispatched', '2026-09-19 14:05:00', 'SENT', 0),
  (8005, 5, 5005, 'SMS', 'Order 3005 dispatched', '2026-09-20 14:05:00', 'SENT', 0);

INSERT INTO audit_logs (log_id, performed_by, entity_type, action_performed, old_value, new_value, performed_at) VALUES
  (9001, 26, 'ORDER', 'CREATE', NULL, 'Order 3001', '2026-09-16 11:05:00'),
  (9002, 27, 'ORDER', 'CREATE', NULL, 'Order 3002', '2026-09-17 11:05:00'),
  (9003, 28, 'ORDER', 'CREATE', NULL, 'Order 3003', '2026-09-18 11:05:00'),
  (9004, 29, 'ORDER', 'CREATE', NULL, 'Order 3004', '2026-09-19 11:05:00'),
  (9005, 30, 'ORDER', 'CREATE', NULL, 'Order 3005', '2026-09-20 11:05:00');
```

Ln 456, Col 28

Output

```
#41: OK, 5 rows affected in 4.776ms
#42: OK, 5 rows affected in 4.916ms
#43: OK, 5 rows affected in 5.003ms
#44: OK, 5 rows affected in 3.352ms
✓ SQL Script execution completed in 303ms. 4 statements executed successfully.
```

2. Verification Queries

```
456     -- PART C: EVIDENCE QUERIES
457   • SHOW TABLES;
458   • SELECT * FROM users;
459   • SELECT * FROM customers;
460   • SELECT * FROM pharmacists;
461   • SELECT * FROM pharmacy_assistants;
462   • SELECT * FROM delivery_couriers;
463   • SELECT * FROM system_administrators;
464   • SELECT * FROM finance_managers;
465   • SELECT * FROM operations_managers;
466   • SELECT * FROM medicines;
467   • SELECT * FROM inventory_batches;
468   • SELECT * FROM prescriptions;
469   • SELECT * FROM prescription_verifications;
470   • SELECT * FROM chronic_refill_subscriptions;
471   • SELECT * FROM orders;
472   • SELECT * FROM order_items;
473   • SELECT * FROM delivery_routes;
474   • SELECT * FROM deliveries;
475   • SELECT * FROM cold_chain_tags;
476   • SELECT * FROM temperature_logs;
477   • SELECT * FROM notifications;
478   • SELECT * FROM audit_logs;
```

3. Created Tables (`SHOW TABLES`)

```
461     • SHOW TABLES;
462
```

Output | Result #45

Tables_in_mediorder_part02_demo_v3
audit_logs
chronic_refill_subscriptions
cold_chain_tags
customers
deliveries
delivery_couriers
delivery_routes
finance_managers
inventory_batches
medicines
notifications
operations_managers
order_items
orders
pharmacists
pharmacy_assistants
prescription_verifications
prescriptions
system_administrators
temperature_logs
users

4. Sample Data Outputs

Users

```
463    • SELECT * FROM users;
```

Output | Result #46

user_id	full_name	phone_number	email	role
1	Sample Customer 1	0770000001	user1@example.test	CUSTOMER
2	Sample Customer 2	0770000002	user2@example.test	CUSTOMER
3	Sample Customer 3	0770000003	user3@example.test	CUSTOMER
4	Sample Customer 4	0770000004	user4@example.test	CUSTOMER
5	Sample Customer 5	0770000005	user5@example.test	CUSTOMER
6	Sample Pharmacist 1	0770000006	user6@example.test	PHARMACIST
7	Sample Pharmacist 2	0770000007	user7@example.test	PHARMACIST
8	Sample Pharmacist 3	0770000008	user8@example.test	PHARMACIST
9	Sample Pharmacist 4	0770000009	user9@example.test	PHARMACIST
10	Sample Pharmacist 5	0770000010	user10@example.test	PHARMACIST
11	Sample Assistant 1	0770000011	user11@example.test	ASSISTANT
12	Sample Assistant 2	0770000012	user12@example.test	ASSISTANT
13	Sample Assistant 3	0770000013	user13@example.test	ASSISTANT
14	Sample Assistant 4	0770000014	user14@example.test	ASSISTANT
15	Sample Assistant 5	0770000015	user15@example.test	ASSISTANT
16	Sample Courier 1	0770000016	user16@example.test	COURIER
17	Sample Courier 2	0770000017	user17@example.test	COURIER
18	Sample Courier 3	0770000018	user18@example.test	COURIER
19	Sample Courier 4	0770000019	user19@example.test	COURIER
20	Sample Courier 5	0770000020	user20@example.test	COURIER
21	Sample Admin 1	0770000021	user21@example.test	ADMIN
22	Sample Admin 2	0770000022	user22@example.test	ADMIN

OK, 35 records retrieved in 1.318ms

Customers

```
465    • SELECT * FROM customers;
```

Output | Result #47

user_id	role	address	date_of_birth	known_allergies
1	CUSTOMER	1 Main Street, Colombo	01/01/1995	NULL
2	CUSTOMER	2 Main Street, Colombo	01/01/1995	NULL
3	CUSTOMER	3 Main Street, Colombo	01/01/1995	NULL
4	CUSTOMER	4 Main Street, Colombo	01/01/1995	NULL
5	CUSTOMER	5 Main Street, Colombo	01/01/1995	NULL

Pharmacists

```
465    • SELECT * FROM pharmacists;
```

Output | Result #48

user_id	role	licence_no	specialization	shift_schedule
6	PHARMACIST	PH-0006	Dispensing	Morning
7	PHARMACIST	PH-0007	Dispensing	Morning
8	PHARMACIST	PH-0008	Dispensing	Morning
9	PHARMACIST	PH-0009	Dispensing	Morning
10	PHARMACIST	PH-0010	Dispensing	Morning

pharmacy_assistants

```
467    • SELECT * FROM pharmacy_assistants;
```

Output | Result #49

user_id	role	packing_station	shift_schedule
11	ASSISTANT	P-11	Morning
12	ASSISTANT	P-12	Morning
13	ASSISTANT	P-13	Morning
14	ASSISTANT	P-14	Morning
15	ASSISTANT	P-15	Morning

delivery_couriers

```
468    • SELECT * FROM delivery_couriers;
```

Output | Result #50

user_id	role	vehicle_licence_no	assigned_zone	availability_status
16	COURIER	DL-0016	Zone 1	AVAILABLE
17	COURIER	DL-0017	Zone 2	AVAILABLE
18	COURIER	DL-0018	Zone 3	AVAILABLE
19	COURIER	DL-0019	Zone 4	AVAILABLE
20	COURIER	DL-0020	Zone 5	AVAILABLE

system_administrators

```
468    • SELECT * FROM system_administrators;
```

Output | Result #51

user_id	role	access_level
21	ADMIN	STANDARD
22	ADMIN	STANDARD
23	ADMIN	STANDARD
24	ADMIN	STANDARD
25	ADMIN	STANDARD

finance_managers

```
471    • SELECT * FROM finance_managers;
```

Output | Result #52

user_id	role	department
26	FINANCE	Finance
27	FINANCE	Finance
28	FINANCE	Finance
29	FINANCE	Finance
30	FINANCE	Finance

operations_managers

```
473    • SELECT * FROM operations_managers;
```

Output | Result #53

user_id	role	department
31	OPERATIONS	Operations
32	OPERATIONS	Operations
33	OPERATIONS	Operations
34	OPERATIONS	Operations
35	OPERATIONS	Operations

Medicines

```
472    • SELECT * FROM medicines;
```

Output | Result #54

medicine_id	medicine_name	therapeutic_category	requires_prescription	is_cold_chain	unit_of_measure
1	Sample Temperature Sensitive A	Therapy A	true	true	vial
2	Sample Temperature Sensitive B	Therapy B	true	true	vial
3	Sample Temperature Sensitive C	Therapy C	true	true	pack
4	Sample Temperature Sensitive D	Therapy D	true	true	box
5	Sample Temperature Sensitive E	Therapy E	true	true	vial

inventory_batches

```
473    • SELECT * FROM inventory_batches;
474
```

Output | Result #55

batch_id	medicine_id	batch_number	manufacture_date	expiry_date	quantity_available	batch_status	shelf_location	min_temp_celsius	max_temp_celsius
101	1	B-101	01/01/2026	01/01/2027	100	AVAILABLE	COLD-1	2.00	8.00
102	2	B-102	02/01/2026	02/01/2027	80	AVAILABLE	COLD-2	2.00	8.00
103	3	B-103	03/01/2026	03/01/2027	120	AVAILABLE	DRY-1	NULL	NULL
104	4	B-104	04/01/2026	04/01/2027	65	AVAILABLE	DRY-2	NULL	NULL
105	5	B-105	05/01/2026	05/01/2027	95	AVAILABLE	COLD-3	2.00	8.00

Prescriptions

```
478    • SELECT * FROM prescriptions;
```

Ln 480, Col 1

Output | Result #56

prescription_id	customer_id	prescription_type	file_url	file_hash_sha256	issue_date	upload_timestamp	verification_status
1001	1	DIGITAL	/demo/prescription_1.pdf	0000000000000000000000000000000000000000000000000000000000000001	09/10/2026	09/11/2026	APPROVED
1002	2	DIGITAL	/demo/prescription_2.pdf	0000000000000000000000000000000000000000000000000000000000000002	09/10/2026	09/12/2026	APPROVED
1003	3	DIGITAL	/demo/prescription_3.pdf	0000000000000000000000000000000000000000000000000000000000000003	09/10/2026	09/13/2026	APPROVED
1004	4	DIGITAL	/demo/prescription_4.pdf	0000000000000000000000000000000000000000000000000000000000000004	09/10/2026	09/14/2026	APPROVED
1005	5	DIGITAL	/demo/prescription_5.pdf	0000000000000000000000000000000000000000000000000000000000000005	09/10/2026	09/15/2026	APPROVED

prescription_verifications

```
482    SELECT * FROM prescription_verifications;
```

Output | Result #57

verification_id	prescription_id	pharmacist_id	action_taken	audit_reason	verification_notes	verified_at
1101	1001	6	APPROVED	Valid sample record	Reviewed in demo	09/11/2026
1102	1002	7	APPROVED	Valid sample record	Reviewed in demo	09/12/2026
1103	1003	8	APPROVED	Valid sample record	Reviewed in demo	09/13/2026
1104	1004	9	APPROVED	Valid sample record	Reviewed in demo	09/14/2026
1105	1005	10	APPROVED	Valid sample record	Reviewed in demo	09/15/2026

chronic_refill_subscriptions

```
484    SELECT * FROM chronic_refill_subscriptions;
```

Output | Result #58

subscription_id	customer_id	medicine_id	frequency_days	next_refill_date	max_refill_count	subscription_status
2001	1	1	30	10/15/2026	12	ACTIVE
2002	2	2	30	10/15/2026	12	ACTIVE
2003	3	3	30	10/15/2026	12	ACTIVE
2004	4	4	30	10/15/2026	12	ACTIVE
2005	5	5	30	10/15/2026	12	ACTIVE

Orders

```
486    SELECT * FROM orders;
```

Output | Result #59

order_id	customer_id	prescription_id	subscription_id	assistant_id	order_status	placed_at	total_amount	payment_method	is_chronic_refill
3001	1	1001	2001	11	DISPATCHED	09/16/2026	100.00	CARD	true
3002	2	1002	2002	12	DISPATCHED	09/17/2026	200.00	CARD	true
3003	3	1003	2003	13	DISPATCHED	09/18/2026	300.00	CARD	true
3004	4	1004	2004	14	DISPATCHED	09/19/2026	400.00	CARD	true
3005	5	1005	2005	15	DISPATCHED	09/20/2026	500.00	CARD	true

order_items

```
489    SELECT * FROM order_items;
```

Output | Result #60

order_id	item_no	medicine_id	batch_id	quantity	unit_price	scanned_serial	dispensed_at
3001	1	1	101	1	100.00	SERIAL-0001	09/16/2026
3002	1	2	102	1	200.00	SERIAL-0002	09/17/2026
3003	1	3	103	1	300.00	SERIAL-0003	09/18/2026
3004	1	4	104	1	400.00	SERIAL-0004	09/19/2026
3005	1	5	105	1	500.00	SERIAL-0005	09/20/2026

delivery_routes

```
491  • SELECT * FROM delivery_routes;
```

Output | Result #61

route_id	zone_name	gps_coordinates	geofence_polygon	estimated_distance_km	estimated_eta_minutes	is_active_zone
4001	Zone 1	6.9,79.8	DEMO_POLYGON	5.00	15	true
4002	Zone 2	6.9,79.8	DEMO_POLYGON	10.00	30	true
4003	Zone 3	6.9,79.8	DEMO_POLYGON	15.00	45	true
4004	Zone 4	6.9,79.8	DEMO_POLYGON	20.00	60	true
4005	Zone 5	6.9,79.8	DEMO_POLYGON	25.00	75	true

Deliveries

```
493  • SELECT * FROM deliveries;
```

Output | Result #62

delivery_id	order_id	courier_id	route_id	dispatched_at	delivered_at	delivery_status	otp_attempts	delivery_otp_hash	proof_photo_url	age_verified	identity_verified
5001	3001	16	4001	09/16/2026	NULL	DISPATCHED	0	NULL	NULL	false	false
5002	3002	17	4002	09/17/2026	NULL	DISPATCHED	0	NULL	NULL	false	false
5003	3003	18	4003	09/18/2026	NULL	DISPATCHED	0	NULL	NULL	false	false
5004	3004	19	4004	09/19/2026	NULL	DISPATCHED	0	NULL	NULL	false	false
5005	3005	20	4005	09/20/2026	NULL	DISPATCHED	0	NULL	NULL	false	false

cold_chain_tags

```
495  • SELECT * FROM cold_chain_tags;
```

Ln 490, Col 1

Output | Result #63

tag_id	medicine_id	delivery_id	min_temp_celsius	max_temp_celsius	max_transport_hours	ice_pack_required	packaging_instruction	manifest_generated_at	age_verification_required	identity_verification_r
6001	1	5001	2.00	8.00	12	true	Use tagged packaging	09/16/2026	false	false
6002	2	5002	2.00	8.00	12	true	Use tagged packaging	09/17/2026	false	false
6003	3	5003	2.00	8.00	12	false	Use tagged packaging	09/18/2026	false	false
6004	4	5004	2.00	8.00	12	false	Use tagged packaging	09/19/2026	true	true
6005	5	5005	2.00	8.00	12	true	Use tagged packaging	09/20/2026	false	false

temperature_logs

```
497  • SELECT * FROM temperature_logs;
```

Output | Result #64

log_id	tag_id	delivery_id	logged_at	temperature_reading	breach_detected	alert_sent	log_stage
7001	6001	5001	09/16/2026	4.10	false	false	IN_TRANSIT
7002	6002	5002	09/17/2026	4.20	false	false	IN_TRANSIT
7003	6003	5003	09/18/2026	4.30	false	false	IN_TRANSIT
7004	6004	5004	09/19/2026	4.40	false	false	IN_TRANSIT
7005	6005	5005	09/20/2026	4.50	false	false	IN_TRANSIT

Notifications

```
499    • SELECT * FROM notifications;
```

Output | Result #65

notification_id	user_id	delivery_id	channel_type	message_body	sent_at	delivery_status	retry_count
8001	1	5001	SMS	Order 3001 dispatched	09/16/2026	SENT	0
8002	2	5002	SMS	Order 3002 dispatched	09/17/2026	SENT	0
8003	3	5003	SMS	Order 3003 dispatched	09/18/2026	SENT	0
8004	4	5004	SMS	Order 3004 dispatched	09/19/2026	SENT	0
8005	5	5005	SMS	Order 3005 dispatched	09/20/2026	SENT	0

audit_logs

```
501    • SELECT * FROM audit_logs;
```

Output | Result #66

log_id	performed_by	entity_type	action_performed	old_value	new_value	performed_at
9001	26	ORDER	CREATE	NULL	Order 3001	09/16/2026
9002	27	ORDER	CREATE	NULL	Order 3002	09/17/2026
9003	28	ORDER	CREATE	NULL	Order 3003	09/18/2026
9004	29	ORDER	CREATE	NULL	Order 3004	09/19/2026
9005	30	ORDER	CREATE	NULL	Order 3005	09/20/2026

Part D – SQL queries & outputs

D1 – Simple SELECT: Cold Chain Medicines

```sql
-- PART D: QUERIES
-- D1 Simple SELECT | Lists medicines requiring special temperature handling.
SELECT medicine_id, medicine_name, is_cold_chain
FROM medicines
WHERE is_cold_chain = TRUE
ORDER BY medicine_id;
```

Output | Result #67

medicine_id	medicine_name	is_cold_chain
1	Sample Temperature Sensitive A	true
2	Sample Temperature Sensitive B	true
3	Sample Temperature Sensitive C	true
4	Sample Temperature Sensitive D	true
5	Sample Temperature Sensitive E	true

This query selects medicines that require cold chain handling. The result shows five medicines marked as cold chain medicines.

D2 – JOIN: Delivery Details

```sql
-- D2 JOIN | Connects a delivery, its order, customer and courier.
SELECT d.delivery_id, o.order_id, c.full_name AS customer_name,
       cr.full_name AS courier_name, d.delivery_status
FROM deliveries d
JOIN orders o ON o.order_id = d.order_id
JOIN users c ON c.user_id = o.customer_id
JOIN users cr ON cr.user_id = d.courier_id
ORDER BY d.delivery_id;
```

Output | Result #68

delivery_id	order_id	customer_name	courier_name	delivery_status
5001	3001	Sample Customer 1	Sample Courier 1	DISPATCHED
5002	3002	Sample Customer 2	Sample Courier 2	DISPATCHED
5003	3003	Sample Customer 3	Sample Courier 3	DISPATCHED
5004	3004	Sample Customer 4	Sample Courier 4	DISPATCHED
5005	3005	Sample Customer 5	Sample Courier 5	DISPATCHED

This query joins the deliveries, orders, and users tables. It shows the order, customer, courier, and delivery status for each delivery. The result contains five deliveries.

D3 – Aggregation: Prescription Verification Summary

```
515 • SELECT COUNT(*) AS total_verifications,
516       SUM(CASE WHEN action_taken = 'APPROVED' THEN 1 ELSE 0 END) AS approved_count,
517       SUM(CASE WHEN action_taken = 'REJECTED' THEN 1 ELSE 0 END) AS rejected_count
518   FROM prescription_verifications;
519
```

Output | Result #72

total_verifications	approved_count	rejected_count
5	5	0

This query counts all prescription verifications and calculates how many were approved or rejected. The result shows 5 verifications: 5 approved and 0 rejected.

D4 – GROUP BY and HAVING: Medicines with Low Stock

```
520 • SELECT m.medicine_id, m.medicine_name,
521       SUM(b.quantity_available) AS total_stock
522   FROM medicines m
523   JOIN inventory_batches b ON b.medicine_id = m.medicine_id
524   GROUP BY m.medicine_id, m.medicine_name
525   HAVING SUM(b.quantity_available) < 100
526   ORDER BY total_stock;
527
```

Output | Result #73

medicine_id	medicine_name	total_stock
4	Sample Temperature Sensitive D	65
2	Sample Temperature Sensitive B	80
5	Sample Temperature Sensitive E	95

This query groups inventory batches by medicine and calculates the total available stock for each medicine. The HAVING condition selects medicines with fewer than 100 units. The result shows three medicines with 65, 80, and 95 units available.

D5 – Subquery: Orders Above the Average Amount

```
510     -- D5 Subquery | Finds orders whose total exceeds the average order total.
511   • SELECT order_id, customer_id, total_amount
512     FROM orders
513     WHERE total_amount > (SELECT AVG(total_amount) FROM orders)
514     ORDER BY total_amount DESC;
```

Output | Result #71

order_id	customer_id	total_amount
3005	5	500.00
3004	4	400.00

This query compares each order amount with the average amount of all orders. The result shows orders 3005 and 3004**, which are above the average.**

Part E – Stored function/procedure

This stored procedure accepts an order ID as input. It combines order,

```sql
DROP PROCEDURE IF EXISTS sp_order_prescription_summary;
CREATE PROCEDURE sp_order_prescription_summary(IN p_order_id INT)
SELECT o.order_id,
       c.full_name AS customer_name,
       m.medicine_name,
       oi.quantity,
       p.verification_status AS prescription_status,
       ph.full_name AS pharmacist_name,
       o.order_status,
       o.total_amount
FROM orders o
JOIN users c ON c.user_id = o.customer_id
LEFT JOIN order_items oi ON oi.order_id = o.order_id
LEFT JOIN medicines m ON m.medicine_id = oi.medicine_id
LEFT JOIN prescriptions p ON p.prescription_id = o.prescription_id
LEFT JOIN prescription_verifications v
    ON v.prescription_id = p.prescription_id
LEFT JOIN users ph ON ph.user_id = v.pharmacist_id
WHERE o.order_id = p_order_id;
```

```
Output
#1: OK, 0 records retrieved in 0.45ms
#2: OK, 0 records retrieved in 4.278ms
#3: OK, 0 records retrieved in 1.88ms
✓ SQL Script execution completed in 133ms. 3 statements executed successfully.
```

customer, medicine, prescription, and pharmacist information to display a summary of the selected order.

Sample execution: `CALL sp_order_prescription_summary(3001);`

```sql
CALL sp_order_prescription_summary(3001);
```

Output | Result #1

order_id	customer_name	medicine_name	quantity	prescription_status	pharmacist_name	order_status	total_amount
3001	Sample Customer 1	Sample Temperature Sensitive A	1	APPROVED	Sample Pharmacist 1	DISPATCHED	100.00

The procedure returned order 3001 with its customer, medicine, approved prescription, verifying pharmacist, and order details.

Part F – Trigger

Trigger: Lock Expired Inventory Batches

```
546     -- PART F: TRIGGER AND DEMONSTRATION
547     DROP TRIGGER IF EXISTS trg_lock_expired_batch;
548
549     CREATE TRIGGER trg_lock_expired_batch
550     BEFORE INSERT ON inventory_batches
551     FOR EACH ROW
552     SET NEW.batch_status =
553         CASE
554             WHEN NEW.expiry_date < CURDATE() THEN 'LOCKED'
555             ELSE NEW.batch_status
556         END;
557
558     INSERT INTO inventory_batches
559     (batch_id, medicine_id, batch_number, manufacture_date, expiry_date,
560      quantity_available, batch_status, shelf_location)
561     VALUES
562     (106, 1, 'B-EXPIRED-TEST', '2024-01-01', '2025-01-01',
563      0, 'AVAILABLE', 'QUARANTINE');
564
565     SELECT batch_id, batch_number, expiry_date, batch_status
566     FROM inventory_batches
567     WHERE batch_id = 106;
568
```

Output | Result #80

batch_id	batch_number	expiry_date	batch_status
106	B-EXPIRED-TEST	01/01/2025	LOCKED

The trigger changed the expired batch status from AVAILABLE to LOCKED before the record was saved.